



PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 3

Introduction to OOPS Programming

SESSION / ASSIGNMENT – 3

TASKS :

1. Create a class called Playlist with a default constructor that sets the playlist name to 'My Favourites' and prints a welcome message when an object is created.



```
#include <iostream>
#include <string>
using namespace std;

class Playlist
{
public:
    string name;

    // Default constructor
    Playlist()
    {
        name = "My Favourites";
        cout << "Welcome to your playlist!" << endl;
    }
};

int main()
{
    Playlist playlist;

    cout << "Playlist Name: " << playlist.name << endl;

    return 0;
}
```

OUTPUT:

```
Welcome to your playlist!
Playlist Name: My Favourites
```

2. Build a class Product for a Flipkart-style app with a parameterized constructor that takes productName, price, and rating as arguments and displays all details using a displayInfo() method.



```
#include <iostream>
#include <string>
using namespace std;

class Product
{
public:
    string productName;
    float price;
    float rating;

    // Parameterized constructor
    Product(string name, float p, float r)
    {
        productName = name;
        price = p;
        rating = r;
    }

    void displayInfo()
    {
        cout << "Product Name: " << productName << endl;
        cout << "Price: " << price << endl;
        cout << "Rating: " << rating << endl;
    }
};

int main()
{
    Product product("Wireless Headphones", 1499.00, 4.5);

    product.displayInfo();

    return 0;
}
```

OUTPUT:

```
Product Name: Wireless Headphones
Price: 1499
Rating: 4.5
```

3. Create a class **Movie** with a parameterized constructor and then use a copy constructor to duplicate a **Movie** object. Print both the original and copied movie details to show they are identical.



```
#include <iostream>
#include <string>
using namespace std;

class Movie
{
public:
    string title;
    int year;

    // Parameterized constructor
    Movie(string t, int y)
    {
        title = t;
        year = y;
    }

    // Copy constructor
    Movie(const Movie &movie)
    {
        title = movie.title;
        year = movie.year;
    }

    void display()
    {
        cout << "Movie: " << title << endl;
        cout << "Year: " << year << endl;
    }
};

int main()
{
    // Create the original movie
    Movie original("Avengers: Endgame", 2019);

    // Create a copy of the original movie
    Movie copy(original);

    cout << "Original Movie:" << endl;
    original.display();

    cout << "\nCopied Movie:" << endl;
```

```
copy.display();

return 0;
}
```

OUTPUT:

```
Original Movie:
Movie: Avengers: Endgame
Year: 2019

Copied Movie:
Movie: Avengers: Endgame
Year: 2019
```

4. Simulate a BookMyShow ticket booking by creating a Ticket class that prints 'Saving your ticket...' in its destructor. Create and delete a Ticket object to demonstrate the destructor lifecycle.

Hint: Use the `__del__` method (Python) or `~Ticket()` (C++) to define the destructor.



```
#include <iostream>
using namespace std;

class Ticket
{
public:

    Ticket()
    {
        cout << "Ticket booked successfully!" << endl;
    }

    // Destructor
    ~Ticket()
    {
        cout << "Saving your ticket..." << endl;
    }
};

int main()
{
    Ticket *ticket = new Ticket();

    cout << "Ticket is being used..." << endl;

    delete ticket;
```

```
    return 0;
}
```

OUTPUT:

```
Ticket booked successfully!
Ticket is being used...
Saving your ticket...
```

5. Modify your Playlist class so that it auto-saves the playlist name to a file called autosave.txt when the object is destroyed, simulating an auto-save feature like Spotify.

Hint: Write the file-saving code inside the destructor.



```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Playlist
{
public:
    string name;

    // Constructor
    Playlist()
    {
        name = "My Favourites";
        cout << "Playlist created." << endl;
    }

    // Destructor
    ~Playlist()
    {
        ofstream file("autosave.txt");

        file << name;

        file.close();

        cout << "Playlist automatically saved." << endl;
    }
};

int main()
{
    Playlist playlist;
```

```
    cout << "Playlist Name: " << playlist.name << endl;  
  
    return 0;  
}
```

OUTPUT:

```
Playlist created.  
Playlist Name: My Favourites  
Playlist automatically saved.
```